Systems Administration (ASIST)

Practical Theoretical Class 03

Permissions and user rights.
User workspaces (*homes*).
Quota control.

Implicit groups

User accounts exist in order to identify different actors in the system and establish what types of access each actor has to the resources made available by the system. That is to establish a distinct set of permissions (*capabilities*) for each user.

The mere fact that the user account exists can immediately establish a base permission set, such as access to the system (login), read access to some objects, or even create objects in temporary folders. These base permissions come from the user automatically belonging to certain ***Implicit Groups*** or ***Special Identities***.

On **Unix** systems the implicit group that stands out is ***others***. All valid users belong to this group.

On **Windows** systems there are several implicit groups where users are automatically placed according to their state, for example:

Everyone – all valid users, including **Guest** and system accounts (Guest user account is disabled by default, allows passwordless access).

Authenticated Users – All users who are authenticated through *username/password*.

Permission assignment

Depending on the type of permissions, they can be associated with users in two ways:

- **Permissions on system objects**: are associated with the object in the form of an ***Access Control List*** (**ACL**). The ACL contains a list of users and groups and for each of them the permissions you have on the object.
- **General system permissions** (***user rights***): are associated with the user account (**or preferably a group account**) through a set of attributes (***user rights***). For example on Windows systems the **Log on locally** permission allows you to log in to the machine console.

Permissions, and in particular general system permissions, should be assigned to users indirectly by making users members of existing groups for the intended purpose, examples:

On Linux systems, members of the **sudo** group are allowed to execute commands as administrators (***root***). Members of the **adm** group have permission to access system logs.

On Windows systems, members of the ***Account Operators*** group have permission to manage user and group accounts.

Permissions on system objects(ACLs)

According to the type of file system and operating system you are using, we have different types of ACLs associated with each object.

Actually the type of ACL is more dependent on the operating system than the file system, when the file system does not contain the appropriate ACL type for the operating system the necessary adaptations (equivalence of permissions) will be made.

- **ACLs on Windows systems:** derived from ACL supported by the NTFS (*New Technology File System*) file system , allow freely specify a list of users and groups and permissions associated with each.

- **ACLs on Unix/Linux systems:** as a base, they use the **POSIX ACLs**, more precisely the POSIX.1e or POSIX.1-2001 (IEEE Std 1003.1-2001) standard, also called ***Single UNIX Specification version 3***. Additionally, the Linux and other Unix family operating systems also support ***extended*** ACLs, are more flexible than previous ones, and more similar to those of Windows systems.

POSIX (***Portable Operating System Interface***) is a set of IEEE standards that aims to maintain some compatibility between operating systems of different types. Native Linux ACLs are POSIX, Windows systems provide a POSIX interface for NTFS ACLs.

Base ACLs (POSIX) of Linux systems

Base/minimum ACLs on Linux systems only allow setting of permissions for **3 entities:**

- the ***owner*** of the object – each object has a unique owner, initially the user who created it. The owner of an object can only be changed by the administrator (root).
- The ***group*** of the object – each object is associated to a user group, initially it is the primary group of the user who created it. A user can change the object's group if he is it's owner and member of the group to which he wants to change.
- Implicit group ***others*** – all valid users.

A user's **effective permissions** are the **sum of the permissions** they obtain through the owner and group entities.

For each of the entities there are 3 permissions: **read**, **write** and **execute**. Although the designations of permissions are reasonably clear, their practical significance varies slightly with the type of object to which the ACL is applied.

POSIX ACL – read, write and execute permissions

Permission	Files	Folders
read	Lets read the contents of the file.	Allows to list the names of objects contained in the folder. Object attributes and permissions are accessible only if you have folder access (execute permission).
write	Allows to change and eventually delete file contents.	Allows to create new objects in the folder, delete existing objects, and change their properties. This is valid only if you have access to the folder (permission execute).
execute	Lets run the file. If the file is a ***script*** (interpreted program), its execution also requires read permission.	**Folder access. To have access to a folder you must have execute permission on all folders from the file system root to the desired folder.**

Remarks: Changing the name or other property of an object or removing the object does not require *write* permission on the object but on the folder where the object is located. Without permission *execute* in a folder all objects in it and subfolders are inaccessible regardless of other permissions.

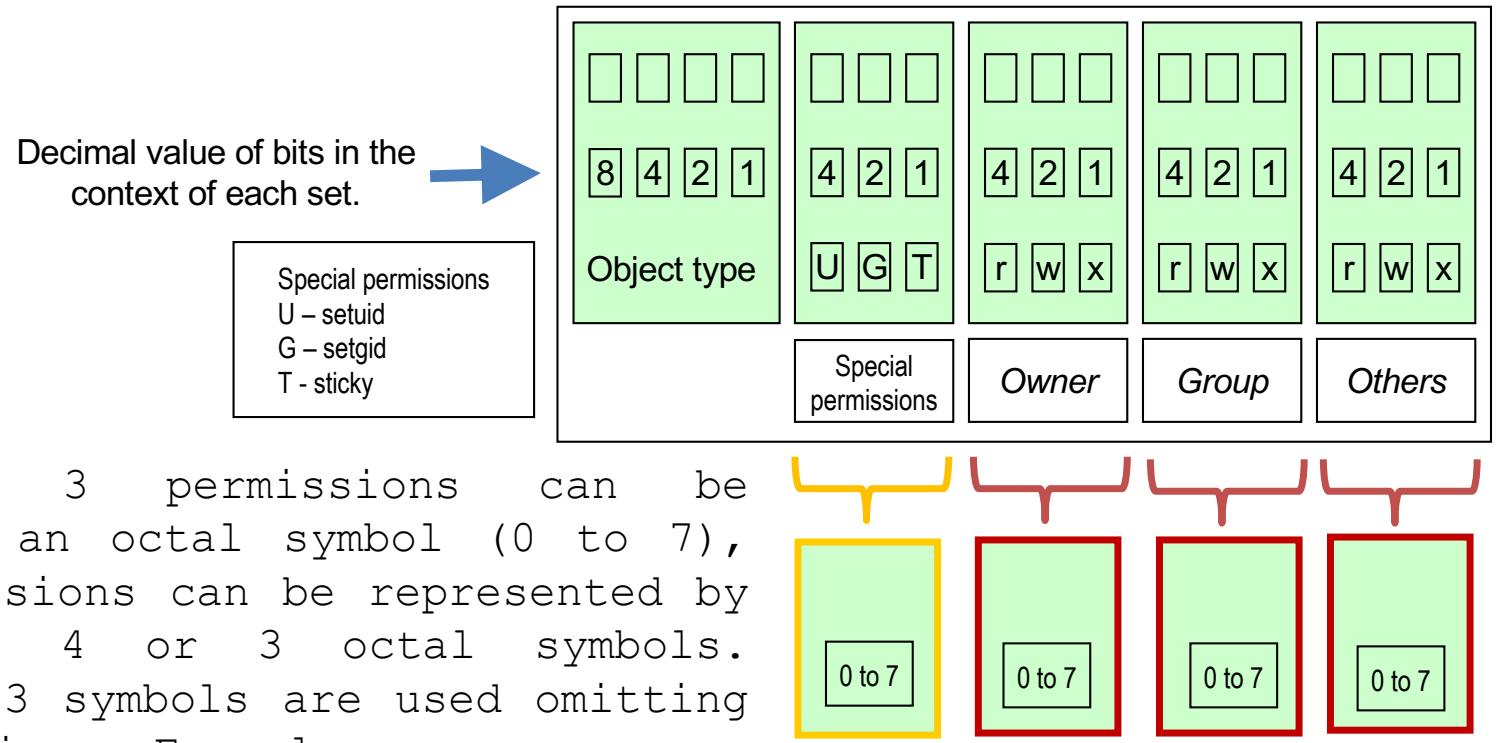
POSIX ACL – setuid, setgid and sticky permissions

The *read*, *write* and *execute* permissions are defined for the *owner*, *group* and *others* (9 bits). In addition to these there are 3 special permissions:

setuid	If the user has the necessary permissions to execute the file then the created process can be executed with the file owner permissions using *system-call* **setuid()**. This expedient is used in commands like ***passwd, sudo*** and ***ping*** for example. These are commands that must perform operations that are not accessible to current users. To assign this permission, the owner of the file/command, typically the ***root*** user, must rely completely on the integrity of the command (only does what it is supposed to do).
setgid	Similar to above, but for group. If the user has the necessary permissions to execute the file then the created process can be executed with the file group permissions using *system-call* **setgid()**.
sticky	Used in shared temporary folders where all users can create objects. Causes ***write*** permission on the folder to be no longer sufficient to remove, rename or change properties of objects stored in the folder, only the owner of the object itself can perform these operations.

POSIX ACL – Octal representation of permissions

In total there are 12 permissions, these permissions are stored in a 16-bit set that is associated with each object as follows:



Each set of 3 permissions can be represented by an octal symbol (0 to 7), thus the permissions can be represented by a sequence of 4 or 3 octal symbols. Typically only 3 symbols are used omitting special permissions. Examples:

700 – all permissions for owner, no permissions for group or others.

644 – read + write permissions for owner, read permission for group or others.

751 – all permissions for owner, read and execute for group, execute for others.

POSIX ACL – initial permissions - UMASK

Object permissions can be viewed with the **ls -l** command and changed with the **chmod** command. Regarding the initial permissions of a newly created object depend on the application that creates the object and on **UMASK** (*user file-creation mode mask*).

UMASK is a **permission denial** mask in octal representation previously described, its meaning is that the bits in UMASK must be **removed** from the permissions of objects created.

One of the most commonly used UMASK is 022 or 0022 indicates that on created objects, write permissions for group and others should always be removed. A UMASK that guarantees complete privacy about created objects is 0077.

At the shell prompt UMASK can be set using the **umask** command, each process/application has an independent UMASK, when a process is created (*fork*) the child process inherits UMASK from the parent process, but can then change it.

In system login scripts (executed in the user's environment after login) the administrator usually includes the command:

umask 0022

If the user does not change this UMASK, it will be inherited by Shell and all applications launched from it.

Windows permissions (NTFS permissions)

Permissions used on Windows Server systems are derived from the NTFS file system and can set permissions for no entities or for a variable number of *Access Control Entries* (**ACE**).

For each ACE (typically a user or group, but may also be a process for example), the ACL defines a permission set. While in POSIX ACLs there are only 3 permissions (*read*, *write* and *execute*) plus 3 special permissions, the different permissions that can be associated with an ACE on a Windows ACL are quite detailed and numerous (***special permissions***).

To facilitate its handling 6 ***basic permission*** sets are defined:

- Full Control
- Modify
- Read & Execute
- List Folder Contents (folders only)
- Read
- Write

Windows – basic permissions and special permissions

	Basic permissions					
Special permissions	**Full Control**	**Modify**	**Read and Execute**	**List Folder Contents (folders only)**	**Read**	**Write**
Traverse Folder/Execute File	YES	YES	YES	YES		
List Folder/Read Data	YES	YES	YES	YES	YES	
Read Attributes	YES	YES	YES	YES	YES	
Read Extended Attributes	YES	YES	YES	YES	YES	
Create Files/Write Data	YES	YES				YES
Create Folders/Append Data	YES	YES				YES
Write Attributes	YES	YES				YES
Write Extended Attributes	YES	YES				YES
Delete Subfolders and Files	YES					
Delete	YES	YES				
Read Permissions	YES	YES	YES	YES	YES	YES
Change Permissions	YES					
Take Ownership	YES					
Synchronize	YES	YES	YES	YES	YES	YES

Windows permissions – inheritance and denial of permissions

On Windows systems permissions are inherited automatically. If a folder has certain permissions, objects that are created in the folder inherit the folder permissions (***inherited permissions***). Inherited permissions are not in the object ACL, but have identical practical effect.

In Windows ACLs each permission can be given (***Allow***) or denied ***(Deny)*** the actual (final) permissions of a user are defined by the sum of the given permissions through the various ACE deleted from the denied permissions through the various ACE. ***Deny* permissions take precedence over *Allow* permissions.**

However, **explicit permissions (set on the object's ACL) take precedence over inherited permissions.** So if the *Deny* of a permission is inherited and in the object's ACL the same permission is given *(Allow)* then that permission will be effective.

Precedences: Explicit Deny - Explicit Allow - Inherited Deny - Inherited Allow

Another important aspect is that Windows ACLs can not only be associated with file system objects such as files and folders but can also be associated with other types of operating system objects such as processes and services.

ACLs Windows – DACL and SACL

In current Windows Server systems there are two types of ACL with different purposes:

DACL (*Discretionary Access Control List*) – is intended to control object access by setting the permissions of each ACE. Each permission can be given (*Allow*) or withdrawn (*Deny*). To manage this type of ACL just have the appropriate permissions on the object.

SACL (*System Access Control List*) – intended to record access (*audit reports*). For each ACE contains a list of permissions that when applied are logged. For each ACE and for each permission, successful and/or failed access attempts can be logged. SACLs can only be managed by administrators.

In Windows all objects have an ***owner***, which can be a user or a group. Initially the owner is the user who creates the object, but it can be changed by having the ***Take Ownership*** permission, or having the user right ***Take ownership of files or other objects***, as with the **Administrators** group.

Regardless of permissions on ACLs on Windows the owner can always change permissions on the object.

Home directory

The concept of home directory is common to most multiuser operating systems, usually the ***home directory*** identification is an attribute stored in the user account, each user must have a separate home directory.

For current users this should be a folder where the user has full permissions and no other user has change permissions. Typically the user is the owner of their ***home directory***.

Being the owner and having full permissions, it is up to the user to manage their content and control their access permissions.

In a network environment the ***home directory*** must be available over the network.

Linux: If there is a set of servers connected to a centralized authentication system, then each user will have the same UID on all systems, in which case the *Network File System* (**NFS**) can be used to mount personal folders on all servers. NFS retains original object owners and permissions.

Windows domain: personal folders can be shared across the network from a domain-owned Windows Server.

Windows domain – *User Profile*

On Windows desktops there is another concept called ***User Profile***, but this is neither the user account nor the user's home directory.

User Profile is a concept behind Windows workstations, it contains user personal *registry* settings (NTuser.dat file) and personal items to put on the *desktop*, *start menu*, etc..

In any workstation there is a ***Default User Profile***, when a user logs in to the workstation, if he does not have a personal *User Profile* one will be created by copying the local *Default User Profile*. User configuration changes and environment customizations are stored in your local personal *User Profile*. The *User Profile* is usually contained in a folder with the username.

When the Windows desktop is a ***domain member***, this means that the user does not log in to the local desktop but in the Windows domain, in which case several options are set regarding the operation of the *User Profile*.

In a Windows domain we can have *User Profiles* defined in the domain itself, for this purpose it is necessary to define the *User Profile* attribute in the user account. This attribute must contain the identification of a shared folder on the network where the profile is located.

Windows domain – *User Profile - scenarios*

A. The *User Profile* attribute is not set on the user account, ie it does not exist. In this case there may be a domain ***Default User Profile*** (***Default User*** folder) on the domain controller's **NETLOGON** network share (the **NETLOGON** share is automatically replicated between all domain DCs).

If there is no personal User Profile on the workstation, one will be created by copying the domain controller's ***Default User Profile*** or, if it does not exist, by copying the local ***Default User Profile*** on the workstation.

In this case the changes made to the profile are merely local, persist in the workstation, but if the user enters another workstation he will have a new profile.

B. The *User Profile* attribute is set on the user account, pointing to a folder shared by a domain server. In the referred folder is an **NTuser.dat** file, so we are facing a ***Roaming User Profile***. Being a *Roaming Profile*, when you log in the profile is copied from the network share to the workstation, when you log out the profile is copied from the workstation to the network share. Obviously the user must have the necessary permissions on the network share. With this type of configuration you always have the same custom profile regardless of the workstation you log in to.

Windows domain – *User Profile - scenarios*

C. The *User Profile* attribute is set on the user account, pointing to a folder shared by a domain server. In the mentioned folder is a file **NTuser.man** (not ***Ntuser.dat***), so this is a ***Mandatory User Profile***. When logging in the profile is copied from the network share to the workstation, logging out the **local profile is deleted**. In this case the *User Profile* attribute can be the same for all users and the share will be read only. With this type of profile the user whenever logging in has the same profile (copied from the share) even on successive logins on the same workstation.

Additionally, if the folder name of ***Mandatory User Profile*** ends with **.man** it will be a ***Super-mandatory User Profile***, the difference is that if planned network sharing is not available then the user is prevented from logging in at all. With a normal ***Mandatory User Profile*** or ***Roaming User Profile***, if sharing is not available scenario **A** is used.

The ***Mandatory User Profile*** alternatives and the use of a domain ***Default User Profile*** have some similarities, except that in the second case the profile is copied from the server only if the user profile does not exist locally, in the first case it is always copied.

Quota control

From the moment users are allowed to write to folders resident in file systems (partitions/volumes) controlled by the administrator, it is essential to maintain control over the space used by each user. That is the mission of quota control.

Without quota control a user could place data in the folder until space available on the corresponding partition/volume. At that time other users and the system itself would no longer be able to write to other folders on the same partition/volume.

Each user's quota is the sum of partition/volume space occupied by all objects owned by that user. Quota control lets you set quota limits, the units used can be multiple: Kb, Mb, Gb. In Linux the unit used is the block whose size can vary between distributions and/or *file systems*.

Usually two quota limits are set: ***soft limit*** and ***hard limit***, the second should be higher than the first. When a user reaches the ***soft limit*** can still use more space but is warned that it is exceeding the quota limit. Reached ***hard limit***, can no longer write. A ***grace period*** can also be defined (maximum time - usually expressed in days - during which it is acceptable to exceed the ***soft limit***, beyond this time the user can no longer use any space above the ***soft limit***).

Quota control – Linux and Windows Server

Linux: quota control is possible for users and/or groups (group quota represents the sum of the spaces occupied by all objects associated with the group). Quotas on Linux are set for each partition, it is not possible to control quotas on an individual folder. In addition to quotas and quota limits related to space occupied, on Linux it is also possible to define *i-node* quotas, ie the number of objects.

Windows Server: There are two independent quota control systems in current versions of Windows Server: ***Disk Quotas*** and ***Directory Quotas***.

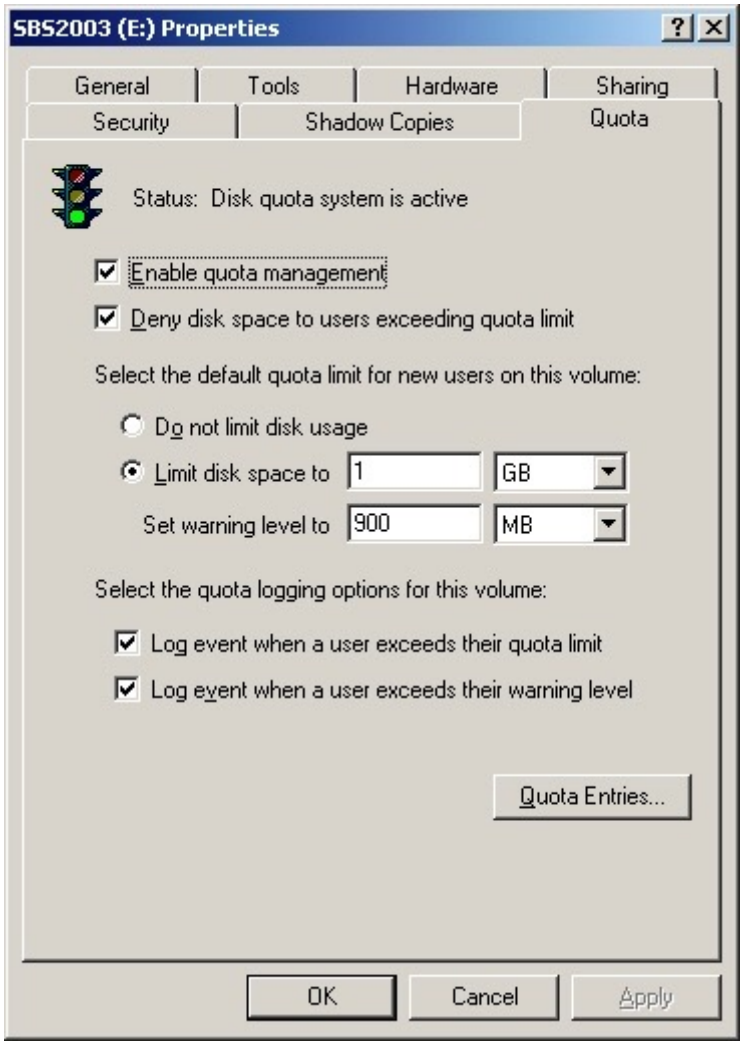
- ***Disk Quotas*** – introduced in *Windows 2000* allow you to control individual user quotas for a volume that is associated with a drive letter, again it is not possible to control the quotas of an individual folder. Group quotas are not supported, in Windows objects are not associated with groups, only users (***owner***).
- ***Directory Quotas*** – introduced in *Windows Server 2003 R2*, allow you to set quota limits for a folder, but this is not a quota for individual users, but all users. This means that one user can occupy all available quota leaving other users with no space.

Quota control – Windows Server

Of course ***Directory Quotas*** do not meet the goal of controlling the space occupied by each user. Numa perspetiva simplista apenas permitem gerar alertas quando o nível de ocupação de espaço de uma pasta pelo conjunto dos utilizadores atinge um dado limite.

Disk Quotas are always drive letter driven (associated with a volume). It is possible to define general limits applied to all users, to consider the space occupied previously it is necessary to execute the **dirquota** command. It is also possible to apply quota limits user to user (*Quota Entries*).

A **network share** of a folder residing on a ***Disk Quotas*** volume is also subject to the quotas set. However, in order to view quota values correctly, users will need to map a drive letter to the network share.



Quota control – Linux

Quota control is performed by the ***kernel*** during file system handling operations, the ***kernel*** records changes made by the user and, depending on variations, updates the user or group quota register.

Quota accounting by ***kernel*** is not performed in absolute value, only changes are accounted for. The starting point is assumed to be a correct quota record.

The first step in enabling quota control is to mount the partition with the **usrquota** and/or **grpquota** options, for this you need to edit the **/etc/fstab** file and mount the partition again (*remount*).

Having the partition mounted with the necessary options, you must do the initial accounting using the **quotacheck** command. This command will traverse the partition's entire file system and account for the space being occupied by each user and/or group. At the root of the partition will be created the files **aquota.user** and/or **aquota.group** (or depending on the version **quota.user** and/or **quota.group**) containing the quota counted.

After accounting has been completed and the files have been created, the **quotaon** command that instructs the ***kernel*** to start monitoring all write operations and keep the files updated accordingly is executed. If recalculation of space is required, the **quotaoff** command should be used and make the initial accounting again.

Quota control – Linux

However, unless there is a serious disk failure, initial quota accounting (**quotacheck** command) will never be needed again. After the first time the control is passed to the ***kernel*** that keeps the log updated.

Initially all users have zero quota limits, which means they have no quota limit.

Quota limits must be set user to used and group to group, for example using the **edquota** or **setquota** commands.

The edquota command invokes a text editor and allows editing of quota limit values, the setquota command receives the limits directly as command line arguments.

In any case there are always four quota limits in the following order:

Block soft limit – soft space limit in units of 1 Kb*

Block hard limit – hard limit of space in units of 1 Kb*

I-nodes soft limit – soft object number limit

I-nodes hard limit – hard limit on number of objects

* By default

Quota control – Linux

Basic **setquota** command syntax for quotas of a user with login name **username:**

```
setquota  username <block-softlimit> <block-hardlimit> <inode-softlimit> <inode-hardlimit> <filesystem>
setquota -p protousername username <filesystem>
```

The four dimension limits are required arguments, if you do not want to impose one of the limits, you must use zero instead. The second version of the presented syntax lets you copy the quota limit settings from user **protousername** to user **username.**

The **quota** command allows the administrator to view the current status of any user's quotas. It also allows a user to view the current status only of their own quotas. As an example (for an administrator seeing user *i999999* quota):

```
root@server# quota i999999
Disk quotas for user i999999 (uid 2687):
     Filesystem  blocks   quota   limit   grace   files   quota   limit   grace
      /dev/sdb1    4228  524288  525000              66       0       0
```

The root administrator can also get a quota status report from all users using the **repquota** command.

Login scripts

Depending on the context in which the user logs in, programs for various purposes may be automatically executed. Although commonly referred to as **login scripts**, they can be any type of program executable in the environment in question.

Linux: one of the attributes of Unix user accounts is the initial program that runs when the user logs in to a terminal. Since this is usually a Shell (command interpreter) this attribute is also often referred to as ***Login Shell***. There are several alternative command interpreters, some of the most common being **bash** (*Bourne-again Shell*) and **csh** (*C Shell*), each of which automatically runs multiple scripts after user login (shell boot files).

Windows: user accounts contain the ***Logon Script*** attribute. This attribute may contain the name of an executable file in the Windows environment, script or not. If set, the file will be executed after user login.

In the context of login to a domain this file must be on the **NETLOGON** network share as this share is automatically replicated between all DCs in the domain.

Linux – Shell boot files

Shell is the initial program that normally runs when a user enters the system through a terminal. Once logged in, all user activities are triggered from Shell, including launching new applications.

Initial Shell loading is therefore the ideal time to perform user environment and application configuration operations that are intended to be performed automatically upon login.

Each shell type uses distinct boot files, the traditional shell **/bin/sh** executes the **/etc/profile** files and then **.profile** in the user's home directory (**~/.profile**).

/Bin/sh is actually simulated by BASH, when invoked as BASH (**/bin/bash**) uses the following boot file sequence: **/etc/profile; ~/.bash_profile; ~/.bash_login; ~/.profile.**

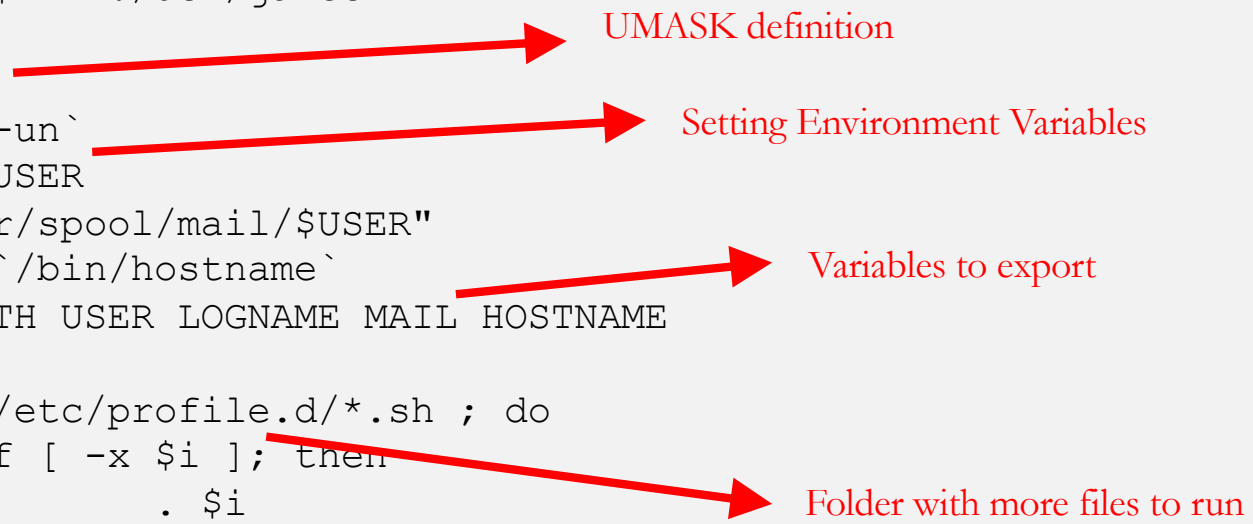
The C Shell, **/bin/csh** or **/bin/csh**, runs the following boot file sequence: **/etc/csh.cshrc; /etc/csh.login; ~/.cshrc; ~/.login.**

One feature to keep in mind is that user area files are executed last, so the user can change what was done in the system files (**/etc/***), although only for him.

Linux -/etc/profile example

```
bash-3.00$ cat /etc/profile
# /etc/profile -*- Mode: shell-script -*-
# (c) MandrakeSoft
if ! echo ${PATH} |grep -q /usr/X11R6/bin ; then
    PATH="$PATH:/usr/X11R6/bin"
fi
if [ "$UID" -ge 500 ] && ! echo ${PATH} |grep -q /usr/games ; then
    PATH=$PATH:/usr/games
fi
umask 022
USER=`id -un`
LOGNAME=$USER
MAIL="/var/spool/mail/$USER"
HOSTNAME=`/bin/hostname`
export PATH USER LOGNAME MAIL HOSTNAME

for i in /etc/profile.d/*.sh ; do
        if [ -x $i ]; then
                . $i
        fi
done
unset i
```



Note that this file is only executed at login for users whose Shell defined in the user account is **/bin/bash** or **/bin/sh.** For C Shell users similar actions will need to be set in the **/etc/csh.cshrc** or **/etc/csh.login** files.

Symbolic links

A symbolic link is a type of object that contains **a pointer to another file system object.** It is supported by some types of file systems and operating systems.

Symbolic links are an important tool for the administrator as they allow folders and objects to be accessed through alternate object locations and names. In other words, they allow objects to appear to have non-real locations and names.

From the point of view of users and applications, symbolic links are transparent, when an application accesses a symbolic link it appears to have the properties of the pointed object (***target***). However, **removing the symbolic link does not remove the pointed object**, only the symbolic link.

For example if on a Windows system there is an **E:\Docs** folder, we can create a symbolic link **C:\Documents** that points to **E:\Docs** (*C:\Documents -> E:\Docs*).

When application users access **C:\Documents** they are actually accessing **E:\Docs**. The apparent properties of **C:\Documents** are the properties of **E:\Docs**.

Windows systems - *Symbolic links*

On Windows systems symbolic links are supported over NTFS file systems from Windows Vista (Workstation) and Windows Server 2008.

Only users with the general user right ***Create Symbolic Links*** can create symbolic links, initially only the *Administrators* group has this permission.

Windows **File Explorer** does not allow you to create symbolic links, from a *File Explorer* point of view symbolic links are interpreted as shortcuts. We should not, however, confuse symbolic links with File Explorer shortcuts. File Explorer shortcuts are normal files that are only recognized by File Explorer and not by other applications.

To create symbolic links in Windows must be used **mklink** command:

mklink [/D] SYMLINK TARGET

If **TARGET** is a folder then the **/D** switch should be used, otherwise something inconsistent is obtained: a file that points to a folder.

SYMLINK and **TARGET** may or may not be on the same file system, volume, or partition. **TARGET** can also be a resident object on a network share.

Unix/Linux systems - *Symbolic links*

On Unix / Linux systems symbolic links can be created by any user who has the necessary permissions on the folder where the link will be created. For this the **ln** command must be used:

ln –s TARGET [SYMLINK]

Regarding the Windows **mklink** command, it is noteworthy that **TARGET** and **SYMLINK** are in reverse order, nor is it necessary to indicate whether TARGET is a folder or not. If **SYMLINK** is omitted a symbolic link with the same name as **TARGET** will be created in the current folder.

Symbolic binding has all permissions active, but this is illusory, the actual permissions are those of the pointed object. Again, removing the symbolic link has no effect on **TARGET**.

If **TARGET** ceases to exist (is removed) symbolic links to it remain but are inoperable (broken).

```
andre@vsrv24:~/TESTE$ ln -s /lib myliblink
andre@vsrv24:~/TESTE$ ln -s /lib2 myliblink2
andre@vsrv24:~/TESTE$ ln -s /etc/passwd
andre@vsrv24:~/TESTE$ ls -l
total 0
lrwxrwxrwx 1 andre profs  4 Set 21 10:08 myliblink -> /lib
lrwxrwxrwx 1 andre profs  5 Set 21 10:08 myliblink2 -> /lib2
lrwxrwxrwx 1 andre profs 11 Set 21  2017 passwd -> /etc/passwd
andre@vsrv24:~/TESTE$
```